Supplementary material

A Proof of Theorem 1

In this section, we give more details on the proof of theorem 1. First, we relate the Lipschitz constant of f to its Jacobian. For differentiable functions, it is well-known that their Lipschitz constant is equal to the supremum over the norm of their Jacobian. As PWL functions generally are not everywhere differentiable, we make use of the generalized Jacobian [Jordan and Dimakis, 2020]:

Definition 3. *The (Clarke) generalized Jacobian of a function f at a point x is defined as the convex hull* $\delta_f(x) := \boldsymbol{co}\{\lim_{x_i \to x} J_f(x_i) : x_i \text{ is differentiable}\}$.

Informally, the generalized Jacobian can be seen as the convex hull of the Jacobians of points close to x in which f is differentiable. In [Jordan and Dimakis, 2020], the following theorem is proven for arbitrary Lipschitz-continuous functions and vector norms, which relates the Lipschitz constant of f to the supremum over the generalized Jacobian:

Theorem 2. *Let $\alpha : \mathbb{R}^d \to \mathbb{R}^m$ be a (p-q)-Lipschitz continuous function on an open set Ω. Let $\delta_\alpha(\Omega) := \bigcup_{x \in \Omega} \delta_\alpha(x)$, then:*

$$L_{p \to q}(\alpha, \Omega) = \sup_{G \in \delta_\alpha(\Omega)} ||G||_{p \to q}. \tag{7}$$

Using this theorem, the proof of theorem 1 can be completed:

Proof. We use the notation of theorem 1 and definition 2. First we argue that α is Lipschitz continuous and that theorem 2 can be applied. We then only need to prove that $\sup_{G \in \delta_\alpha(\Omega)} ||G||_{p \to q} = \max_{i=1,\dots,k} ||T^i||_{p \to q}$. For this, let $x \in \Omega$ and $\widetilde{G} \in \delta_\alpha(x)$ be arbitrary. Since $\delta_\alpha(x)$ is a convex set per definition and since all norms are both continuous and convex, Bauer's maximum principle implies that $\max_{G \in \delta_\alpha(x)}$ occurs on some extreme point of $\delta_\alpha(x)$. Since $\delta_\alpha(x)$ is defined as a convex hull (see def. 3), there must exist a sequence $(x_i)_{i \in \mathbb{N}}$ such that $||\lim_{x_i \to x} J_\alpha(x_i)|| \geq ||\widetilde{G}||$. Since, for an arbitrary index i, the vector x_i is in the interior of exactly one polyhedron $\mathcal{P}_{j_i}$ from the PWL decomposition of α, it holds that $J_\alpha(x_i) = T^{j_i}$. Therefore, it holds that $||J_\alpha(x_i)|| \leq \max_{i=1,\dots,k} ||T^i||$. Since norms are continuous functions, it therefore follows that

$$||\widetilde{G}|| \leq ||\lim_{x_i \to x} J_\alpha(x_i)|| \leq \max_{i=1,\dots,k} ||W_i||_{p \to q} \quad .$$

Since x and $\widetilde{G}$ were chosen arbitrarily, this proves that $\sup_{G \in \delta_\alpha(\Omega)} ||G||_{p \to q} = \max_{i=1,\dots,k} ||T^i||_{p \to q}$. □

B Proof of Equation (3)

In this section, we prove equation 3 using the notation introduced there.

Proof.
"⇒": Let $y \in (W_1 \cdot \Omega_\rho + w_1) \cap \mathcal{P}_i$. Then, because $y \in (W_1 \cdot \Omega_\rho + w_1)$, there must exist an $x \in \Omega_\rho$ such that $W_1 \cdot x + w_1 = y$. Also, because $y \in \mathcal{P}_i$, it must fulfill $A_i \cdot y < a_i$. Inserting the first equation into the second, we see that x fulfills $A_i \cdot W_1 \cdot x < a_i - A_i \cdot w_1$. As $x \in \Omega_\rho$, it also holds that $Cx \leq c$, which provers the rightwards direction.

"⇐": Let $x \in \{x \in \mathbb{R}^{d_0} : \widetilde{A}_i \cdot x \leq \widetilde{a}_i\}$. Then, per definition of $\widetilde{A}_i$ and $\widetilde{a}_i$, it holds that $C \cdot x \leq c$, meaning $c \in \Omega_\rho$. Also, let $y := \widetilde{B} \cdot x + \widetilde{b}$. Per definition of x, it holds that $A_i \cdot \widetilde{B} \cdot x \leq a_i - A_i \cdot \widetilde{b}$, meaning $A_i \cdot y \leq a_i$, proving the leftwards direction. □

C Proof of Lemma 1

As was mentioned in section 3.3, the proof for lemma 1 that was given in [Bhowmick *et al.*, 2021] easily transfers to the general case of $||.||_{p \to q}$-norms, which we show in this section. We therefore explicitly mention that the proofs in this section are only slight variations of the proofs in [Bhowmick *et al.*, 2021]. First, we formulate a helper lemma:

Lemma 3. *Let A and B be two $(m \times n)$-dimensional matrices such that componentwise:*

$$|A_{ij}| \leq B_{ij} \qquad \forall i, \quad .$$

It then holds that $||A||_{p \to q} \leq ||B||_{p \to q}$.

Proof. It is a well-known result from functional analysis that, because $\{x \in \mathbb{R}^n : ||x||_p = 1\}$ is a compact set, there exists an $x^* \in \mathbb{R}^n$ such that $||x||_p = 1$ and $||Ax^*||_q = ||A||_{p \to q}$. Define now a vector x' such that $x'_i = |x^*_i|$ for all i. From the definition of a p vector norm, one can easily see that $||x'||_p = ||x^*||_p = 1$. Now, the chain of inequalities holds for $1 \leq i \leq m$:

$$|(Ax^*)_i| = \big|\sum_{j=1}^{n} A_{ij} x^*_j\big| \leq \sum_{j=1}^{n} |A_{ij}| \cdot |x^*_j|$$
$$\overset{|A_{ij}| \leq B_{ij}}{\leq} \sum_{j=1}^{n} B_{ij} \cdot |x^*_j| = (Bx')_i \quad .$$

From the definition of the p vector norms, it is now easy to see that the above inequalities imply $||Ax^*||_q \leq ||Bx'||_q$. We can therefore, due to the definition of x^*, form the following chain of equalities:

$$||A||_{p \to q} = ||Ax^*||_q \leq ||Bx'||_q \leq ||B||_{p \to q} \cdot ||x'||_p = ||B||_{p \to q}$$

□

Having proven lemma 3, the proof of lemma 1 is almost finished.

Proof of lemma 1. For differentiable functions, it is well known that $L_{p \to q}(f, \Omega) = \sup_{x \in \Omega} ||J_f(x)||_{p \to q}$, which together with lemma 3 would prove the result. However, we assume a PWL (and not necessarily differentiable) f. In [Bhowmick *et al.*, 2021], the authors cite [Jordan and Dimakis, 2020], to get a similar result for not everywhere differentiable functions. To our knowledge, [Jordan and Dimakis, 2020] however only prove such a result in the case of general position ReLU networks. As we only require a proof for PWL functions f, we refer therefore to theorem 1, which together with lemma 3 also proves lemma 1. □

D Details on Practical Applications

In this section, we describe implementation details of the NNs for which we computed the Lipschitz constants in the main paper. The weights and biases of all NNs can also be found in the supplementary material. Generally, we used the default initialization by Pytorch, meaning the weights of all linear layers were initialized as samples from a uniform distribution $\mathcal{U}(-\sqrt{\frac{1}{k}}, \sqrt{\frac{1}{k}})$, where k is the number of input features into the corresponding hidden layer. We relied on early stopping (described for each application below) for regularization.

D.1 Absolute Value Function

For the simulations in Table 1, we generated for 20 different random seeds 2000 equidistant points in the interval $[-1, 1]$. For each point, the absolute value was computed and a random noise from a normal distribution $\mathcal{N}(0, 0.1)$ was added. Then, a $(0.64 - 0.16 - 0.2)$ train-validation-test split was performed and NNs with the listed activation functions and shape $[1, 6, 6, 1]$ were trained with the SGD optimizer with a learning rate of 0.1, MSE-loss and batch size 256 to a maximum of 2000 epochs with early stopping with a patience of 20 epochs and a minimum improvement of 10^{-6}.

D.2 Bike Sharing

On the bike sharing data [Fanaee-T and Gama, 2014], we first converted the "date" into a "YYYYMMDD"'-coded integer and then normalized all input features. We then split the data into a $(0.64 - 0.16 - 0.2)$ train-validation-test split.

For the NNs listed in Table 5, we used three target variables: "casual", "registered" and "cnt" and trained ReLU and GroupSort networks of shape $(13, 20, 16, 8, 3)$ with the Adam optimizer with a learning rate of 0.001 and weight decay of 10^{-3} with the MSE loss. We trained for a maximum of 2000 epochs with a batch size of 256 and performed early stopping with a patience of 5 and a minimum improvement of 10^{-6}. The ReLU network achieved a test RMSE of 40.33. The GroupSort network with group size 2 achieved a test RMSE of 40.38.

For the NNs listed in Table 6, we used only the single target variable "cnt", equivalent to the total count of rental bikes with the listed NN shapes and using the same hyperparameters as in Table 5 otherwise.

D.3 Wine Quality Dataset

On the wine quality dataset [Cortez *et al.*, 2009], we dropped the feature "color" and used "quality" as the target variable. The 11 remaining input features were again normalized and a $(0.64 - 0.16 - 0.2)$ train-validation-test split was performed. We then trained the networks listed in Tables 3 and 4 using the Adam optimizer with a learning rate of 0.001, a batch size of 256 to a maximum of 1000 epochs with a patience of 5 and a minimum improvement of 10^{-6} and the MSE loss.

The network of shape $[11, 12, 12, 1]$ achieved a test RMSE of 0.703 with the ReLU activation and 0.712 with the GroupSort activation with group size 2. The network of size $[11, 24, 24, 1]$ achieved a test RMSE of 0.712 for the ReLU- and 0.692 for the GroupSort activation.

D.4 Abalone

For the abalone dataset [Warwick *et al.*, 1994], the categorical "Sex" feature was one-hot encoded and the reference dropped. The data was then normalized and we again used a $(0.64 - 0.16 - 0.2)$ train-validation-test split of the data.

We then trained ReLU and GroupSort networks of shape $(9, 16, 16, 1)$ from Table 2 with the Adam optimizer with a learning rate of 0.001 with the MSE loss for a maximum of 1000 epochs with a batch size of 64 and with early stopping with a patience of 5 and a minimum improvement of 10^{-6}. The ReLU network achieved a test RMSE of 2.14, the GroupSort network one of 2.14 with a group size of two (MaxMin activation).

E Common Activation Functions

In this section, we discuss results pertaining to commonly used activation functions and also describe the polyhedron decomposition we use in our implementation of these functions.

E.1 Componentwise PWL Functions

In this subsection, we show that an activation function that applies a componentwise one-dimensional linear spline, such as ReLU, to the pre-activation is a PWL function in the sense of remark 1. To keep notation simple, we will only discuss settings in which the spline is the same on all dimensions, but our treatment can be easily transferred to the general case.

Let $\alpha : \mathbb{R} \to \mathbb{R}, x \mapsto \sum_{j=1}^{k} (a_j x + b_j) \mathbb{1}_{I_j}(x)$, where I_j are disjunct intervals whose union is $\mathbb{R}$ and where a_j and b_j are appropriate coefficients. It can be easily seen, that this structure includes ReLU for $a_1 = 0, b_1 = 0, I_1 = (-\infty, 0]$ and $a_1 = 1, b_1 = 0, I_1 = (0, \infty)$. It also includes LeakyReLU, simple learned activations like ParametricReLU and complex ones like learned spline activations.

We now assume that $\widetilde{\alpha} : \mathbb{R}^d \to \mathbb{R}^d$ is a componentwise activation function that applies α to each component of the pre-activation. Let then for $i = 1, \ldots, d$ and $j = 1, \ldots, k$ be $\mathcal{P}_{ij}$ be the polyhedron defined by:

$$\mathcal{P}_{ij} = \mathbb{R}^{i-1} \times I_j \times \mathbb{R}^{d-i} \quad ,$$

where we omit $\mathbb{R}^0$ when forming the product. We now claim that the seet $\{\mathcal{P}_{ij}\}_{\substack{i=1,\ldots,d \\ j=1,\ldots,k}}$ is a decomposition as in remark 1. This is easy to see, as, for an arbitrary neuron n, the union $\bigcup_{j=1,\ldots,k} \mathcal{P}_{nj}$ is equal to $\mathbb{R}^d$ and on each $\mathcal{P}_{nj}$, the state of neuron n is fixed, where T^{nj} is a zero-vector except for the entry a_j at its n-th position and $t^{nj} = b_j$.

This is the generalization of the hyper-rectangle representation that [Bhowmick *et al.*, 2021] used.

E.2 GroupSort

In this subsection, we construct the polyhedron decomposition of the GroupSort activation function that was introduced in [Anil *et al.*, 2019] and further studied e.g. in [Tanielian and Biau, 2021]. A GroupSort layer with group size γ will split the pre-activation into groups of size γ, sort the entries of each group in ascending order and return the results.

We also prove mathematically that this is a PWL function and construct an appropriate decomposition of our method. A study of the relation between GroupSort and other PWL functions can also be found in [Tanielian and Biau, 2021].
Our treatment naturally caries over to MaxMin and FullSort, which are special cases of GroupSort with either one group only or $\gamma = 2$ as [Anil *et al.*, 2019] mention.
As a starting point, we discuss the FullSort activation, as the GroupSort activation is essentially just composed of multiple FullSort functions of lower dimensionality. Let therefore $\alpha : \mathbb{R}^\gamma \to \mathbb{R}^\gamma$ be the FullSort activation, i.e. the function that transforms any vector $(x_1, \dots, x_\gamma)$ into a vector with all entries sorted in ascending order:

$$\alpha((x_1, \dots, x_\gamma)) = (x_{i_1}, \dots, x_{i_\gamma}), \tag{8}$$

with $x_{i_1} \leq x_{i_2} \leq \dots \leq x_{i_\gamma}$.

Lemma 4. *FullSort is a PWL function.*

Proof. Let $\pi = (j_1, \dots, j_d)$ be an arbitrary ordering of $1, \dots, \gamma$ (i.e. a *permutation*) and let A be the $(\gamma - 1) \times \gamma$-matrix defined by:

$$A_{kl} = \begin{cases} 1 & \text{, if } l = j_k \\ -1 & \text{, if } l = j_{k+1} \\ 0 & \text{, else} \end{cases} \tag{9}$$

and let b be a zero-vector of length $\gamma - 1$. Then the following holds:

$$\{x \in \mathbb{R}^\gamma : x_{j_1} \leq \dots \leq x_{j_\gamma}\} = \{x \in \mathbb{R}^\gamma : Ax \leq b\} =: \mathcal{P}_\pi$$

and it is easy to see that we can decompose $\mathbb{R}^\gamma$ into such polyhedra. For any $x \in \mathcal{P}_\pi$, we can now compute α:

$$\alpha(x) = (x_{j_1}, \dots, x_{j_\gamma}), \tag{10}$$

which can be equivalently described as a transformation with a $(\gamma \times \gamma)$-permutation matrix P given by:

$$P_{kl} = \begin{cases} 1 & \text{, if } l = j_k \\ 0 & \text{, else} \end{cases}. \tag{11}$$

We have therefore constructed a decomposition of $\mathbb{R}^\gamma$ into $(\gamma!)$-many polyhedra and, constrained to each polyhedron, α is a linear function. □

Let now $\tilde{\alpha} : \mathbb{R}^d \to \mathbb{R}^d$ be the GroupSort activation function with group size γ, which splits the pre-activation into k different groups of size γ, sorts the elements of each group in ascending order and combines the result as its output. We assume that γ is a divider of d, i.e. that $k \cdot \gamma = d$, but our discussion in this chapter as well as our implementation would easily translate to groups of inhomogeneous size.
Using earlier notation, let α again be the FullSort activation operating on γ-dimensional space. For any neuron with index $i \leq d$, we can decompose the index as $i = l_1 \cdot \gamma + l_2$ and write:

$$\tilde{\alpha}_i = \alpha((x_{l_1\gamma+1}, \dots, x_{(l_1+1)\gamma}))_{l_2}, \tag{12}$$

which formalizes the intuition that GroupSort is essentially just a combination of multiple lower-dimensional FullSort functions.

It is therefore not difficult to see that one can construct a decomposition of a form as in definition 2 of $\mathbb{R}^d$ into polyhedra by forming all possible combinations of k polyhedra from decompositions of $\mathbb{R}^\gamma$. Similarly, as can be seen from equation (10), in any such polyhedron, the state of all d neurons would be fixed affine.
Similar to section E.1, we however instead construct a decomposition following remark 1 in which only the activation state of a subset of neurons is fixed in each polyhedron. For this, let $\mathcal{P}_1, \dots, \mathcal{P}_N$ be a decomposition of $\mathbb{R}^\gamma$ into polyhedra, such as the one constructed in the proof of lemma 4. We then define the following set of polyhedra:

$$\mathcal{Q}_{ij} := \mathbb{R}^{(i-1)\gamma} \times \mathcal{P}_j \times \mathbb{R}^{(k-i)\gamma}$$

for $i = 1, \dots, k$ and $j = 1, \dots, N$, where $\mathbb{R}^0$ is ignored when forming the cartesian product.
Then each $\mathcal{Q}_{ij}$ is a polyhedron of dimensionality $\gamma \cdot k = d$ and $\bigcup_{\substack{i \leq k \\ j \leq N}} \mathcal{Q}_{ij} = \mathbb{R}^d$.
Also, on each $\mathcal{Q}_{ij}$, the state of the neurons with indices $(i-1)\gamma + 1, \dots, i\gamma$ are fixed with their linear activation states given by the one of FullSort on $\mathcal{P}_j$.

E.3 Activation Function Complexity

As we discuss below in Section F, the number of polyhedra in the decomposition of a PWL NN is closely linked to the computational cost of ExLipBaB on that NN. As is for example shown in [Goujon *et al.*, 2024], the complexity of the activation function in that NN is a crucial factor in estimating this number, where in this context "complexity" is defined as the number of regions in the PWL decomposition of the activation function. Therefore, given two NNs of equal architecture that differ only in the complexity of their activation function, the NN with the higher complexity activation function is likely to be associated with a higher computational cost of ExLipBaB.

This increase in computational cost can be well illustrated for our implementation in algorithm 2 given the computations in the proofs of section E.2 for a GroupSort-activated network with group size γ. In the `for`-loop in line 5 of algorithm 2, the branching step checks for a non-empty intersection for all polyhedra in the decomposition of the activation function at the current $\star$-neuron. For the polyhedron decomposition of GroupSort from section E.2 this means the body of the loop runs $\gamma!$ times. If a branching step is necessary for all neurons of a layer of size $d = k \cdot \gamma$, then k such loops have to run, as each branching step sets the state of all neurons in a single group. Therefore, the code in algorithm 2 line 6 ff. needs to be run

$$\gamma! \cdot k = \gamma! \cdot \frac{d}{\gamma} = (\gamma - 1)! \cdot d \tag{13}$$

times. This means that the computational cost of setting a single layer potentially increases with the factorial of group size. For a further discussion of the complexity of common PWL activation functions, we refer to [Goujon *et al.*, 2024].

F Scalability

We mentioned in the introduction that the exact computation of a NN's Lipschitz constant has been shown to be an NP-

hard problem. In this section, we will discuss how certain hyperparameters of the NN generally influence computational cost, but leave rigorous (and tighter) convergence speed guarantees for future research.

First, remember that our algorithm functions as a maximization of the Jacobian norm over k polyhedra in the decomposition of the NN f (see thm. 1). The advantage of the Branch-and-Bound framework is that it usually does not need to explore the entire search space of size k, but only a fraction of it. Convergence speed is therefore closely related to both the size k of the search space and the fraction of it which the BnB-algorithm needs to enumerate.

Estimation of k, i.e. the number of linear regions of a PWL NN is currently an active ongoing area of research, as it can also be used as a measure of NN expressiveness. Generally, the exact computation of k has also been shown to be NP-hard [Stargalla *et al.*, 2025; Wang, 2022] but different bounds exist. Often, such bounds quantify the maximum number of linear regions a PWL NN can theoretically reach and [Goujon *et al.*, 2024] show for a wide class PWL NNs that such bounds are generally exponential in network depth and polynomial in network width and activation function complexity (see Supplement E.3). Generally, we would expect this to also be reflected in the corresponding computation time for ExLipBaB.

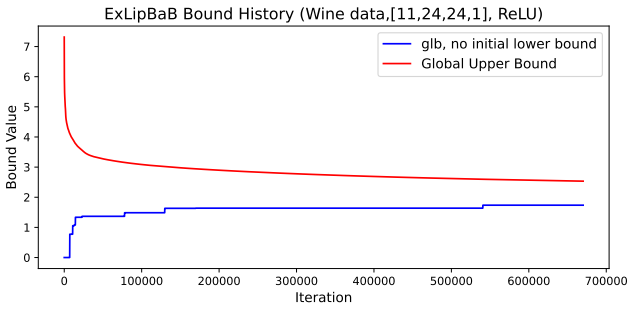
However, we also feel that it is important to mention that it has also been shown both theoretically and in practice that the expected/practical number of linear regions is often much smaller and does not necessarily grow exponentially in depth [Goujon *et al.*, 2024; Hanin and Rolnick, 2019, Thm. 3]. Furthermore, the dependency on activation complexity will be also dependent on the PWL decomposition used for the activation function. For example, we showed in section E.3 that for GroupSort NNs the cost of branching a single layer increases in the factorial of their group size γ using the PWL representation from section E.2. A more suboptimal decomposition into convex affine regions would instead increase with the number of linear regions of GroupSort, which, as [Goujon *et al.*, 2024] show increases with $(\gamma!)^k$, where k is the number of groups in the layer.

Even if the formula from [Goujon *et al.*, 2024, Thm. 2] combined with an empiric runtime over a few iterations of ExLipBaB could therefore easily be used to get a worst-case guarantee for ExLipBaB conversion, such bounds would likely be conservative to the point of unhelpfulness. Also, further research would be needed to translate the cardinality of the state space into a tight estimate for the size of the BnB search tree, requiring also either further study of the tightness of the layerwise upper bound for the Lipschitz constant or the introduction of an alternative bound.

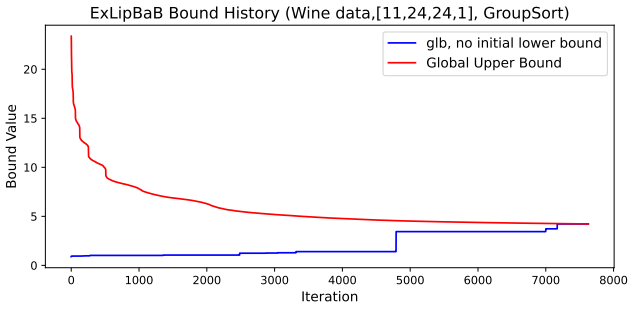
F.1 Empirical Convergence Behavior

To give readers an intuition for practical convergence behavior, we show in figure 2 and 3 the bound histories for the ExLipBaB computations from section 4 on the larger NNs which were trained on the Wine and Bike Sharing data. We selected these specific examples, as they were the ones with the longest runtime and therefore likely most relevant.

We note that for these convergence plots we re-ran our

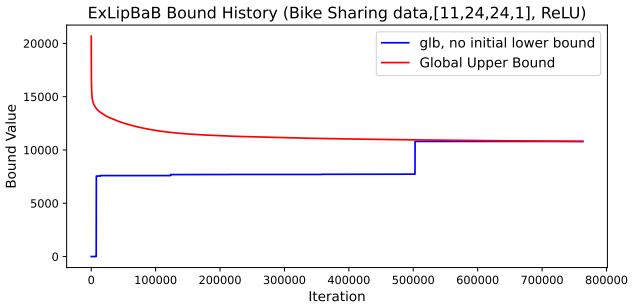


(a) No convergence after 12 hours; final gub: 2.53, final glb: 1.74

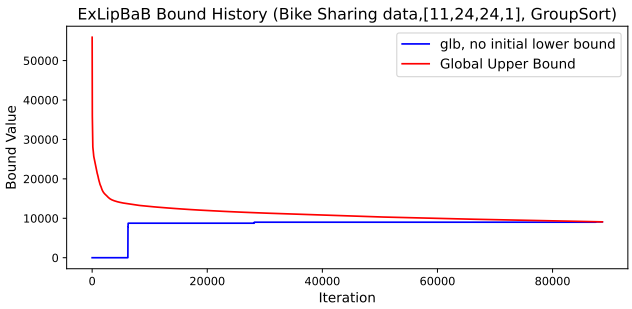


(b) Convergence after 314 seconds, computed constant: 4.23

Figure 2: History of greatest upper/lower bound for ExLipBab computations in table 4.



(a) Convergence after 26404 seconds, computed constant: 10802.4



(b) Convergence after 1832 seconds, computed constant: 9095

Figure 3: History of greatest upper/lower bound for ExLipBab computations in table 5.

computations (with the same setup and weights as in section 4). This will obviously not change the resulting Lipschitz constant or the number of iterations necessary, which we show by listing again the Lipschitz constants obtained in the re-run. The computation times on the other hand also depend on other factors, such as background tasks in the operating system, and differ slightly; crucially in this repeat testing, the GroupSort Bike Sharing NN also converged before cutoff time.

G Code Listings

In this section, we show pseudocode for the branch function as well as a more detailed pseudocode of the main algorithm than the on eprovideed in listing 1.

Algorithm 2 Branch

Input:

- sub-problem ρ
- old glb
- list of weights and biases of Neural net
- α list of instances of PWL
- pnorm

Output:

- new_subproblems: list of instances of SubProblem class
- glb: float, new lower bound on Lipschitz constant over all sub-problems

initialize: new_subproblems $\leftarrow$ empty list
$\star$-neuron_indices $\leftarrow \rho.\star$-neurons$[\rho.\tilde{l}]$
neuron_to_branch $\leftarrow \star$-neuron_indices$[0]$ //branch at first $\star$-neuron
possible_polyhedra $\leftarrow \alpha[\rho_{\tilde{l}}]$.get_polyhedron_ decompostition(neuron_to_branch)
for polyhedron in possible_polyhedra **do**
    intersection $\leftarrow$ join constraints of Ω_ρ and polyhedron backwards propagated with ρ.propagated_weights
    if intersection is empty **then**
        go to next polyhedron
    end if
    act_state_$[\Lambda]$, act_state_$[\lambda] \leftarrow \rho.[\Lambda], \rho.[\lambda]$
    for known decided_neuron **in** polyhedron **do**
        update layer $\tilde{l}$ of act_state_$[\Lambda]$, act_state_$[\lambda]$
    end for
    create subproblem $\rho_{new} \leftarrow$ SubProblem(polyhedron, act_state_$[\Lambda]$, act_state_$[\lambda]$)
    $\rho_{new}.\tilde{l} \leftarrow \rho.\tilde{l}$
    ρ_{new}.star_neurons $\leftarrow \rho$.star_neurons $\setminus$ known decided_neurons
    $\rho_{new} \leftarrow \rho_{new}$.FFilterProp(weights, α) //update the activation pattern and get $\tilde{l}$
    $L_{ub} \leftarrow$ LipschitzBounds(ρ_{new}, $(\widetilde{W}, \widetilde{b})$) //see section 3.3
    new_subproblems $\leftarrow$ new_subproblems.append(ρ_{new})
    if $\rho_{new}.\tilde{l} == L + 1$ **then**
        //no $\star$-neurons, one linear region
        glb $\leftarrow \max($glb$, L_{ub})$
    end if
end for
return new_subproblems, glb

Algorithm 3 Main Algorithm

Input:

- Network $f = \alpha^L \circ \mathfrak{W}^L \circ \ldots \alpha^1 \circ \mathfrak{W}^1$ (list of alternating tuples (W^l,w^l) and instances of PWL α^l)
- input domain Ω (polyhedron)
- approximation factor θ (float)
- p, deciding which p-norm to use

Output:

- (glb, gub) tuple of greatest lower bound and greatest upper bound for $L(f, \Omega)$ where gub $\leq k \cdot$ glb

initialize constraints $H \leftarrow \Omega \subset \mathbb{R}^{n_0}$ //(polyhedron)
weights_list, act_list $\leftarrow f$ //decompose f into weights and activations lists
initial activation pattern ([Λ], [λ], $\tilde{l}$, star_neurons) $\leftarrow$ SymProp(Ω, weights_list, act_list)
$\rho_I \leftarrow$ SubProblem(H, [Λ], [λ]) // instance of SubProblem class
$\rho_I.\tilde{l} \leftarrow \tilde{l}$
ρ_I.star_neurons $\leftarrow$ star_neurons
ρ_I.propagated_weights $\leftarrow$ LinProp(Id, **0**, weights_list, [Λ], [λ], start=0, end =$\rho_I.\tilde{l}$)
$\rho_I.L_{ub} \leftarrow$ LipschitzBounds(ρ_I, weights_list, p) //see seection 3.3
gub, glb $\leftarrow (\rho_I.L_{ub}, 0)$ // initialize upper and lower bound on Lipschitz constant
initialize: $\varrho \leftarrow$ maxHeap([ρ_I]) //(initialize maxheap of subproblems)
if $\tilde{l}$ == L **then**
  //trivial case
  glb $\leftarrow \rho_I.L_{ub}$
end if
while gub $> \theta \cdot$ glb **do**
  $\rho' \leftarrow \arg\max_{\rho \in \varrho}(\rho.L_{ub})$
  $\varrho \leftarrow \varrho \backslash \{\rho'\}$
  (new_subproblems, glb) $\leftarrow$ Branch(ρ', glb, weights_list, act_list, pnorm)
  for $\tilde{\rho}$ in new_subproblems **do**
    if $\tilde{\rho}.L_{ub} > glb$ **then**
      $\varrho \leftarrow (\varrho \cup \{\tilde{\rho}\})$
    end if
  end for
  gub $\leftarrow \max_{\rho \in \varrho} \rho.L_{ub}$
end while
return (glb, gub)